

Parcial 1

Valentina Montenegro

10 de marzo de 2026

1. Primer punto

El primer código implementa un autómata finito determinista cuyo objetivo es verificar si una cadena cumple con un formato específico. La estructura consiste en una secuencia inicial de letras minúsculas, seguida de un operador especial, y finalmente otra secuencia de letras minúsculas que puede terminar con uno o varios dígitos.

Para lograr esto, el programa define un conjunto de estados que representan las diferentes etapas del análisis de la cadena. El autómata comienza en un estado inicial donde espera encontrar letras minúsculas. Si la cadena cumple con esta condición, el autómata permanece en ese estado mientras sigan apareciendo letras.

Posteriormente, cuando aparece un símbolo que indica una transición, como `-` seguido de `>` o el símbolo `X`, el autómata cambia de estado para indicar que ha detectado la parte central del patrón. A partir de ese momento, comienza a analizar la segunda parte de la cadena.

En esta segunda parte, el autómata permite nuevamente letras minúsculas y, opcionalmente, dígitos al final. Si la cadena termina en un estado que pertenece al conjunto de estados de aceptación, entonces la cadena se considera válida. De lo contrario, si aparece un carácter que no corresponde con la estructura esperada, el autómata pasa a un estado de error.

2. Segundo punto

El segundo programa también utiliza un autómata finito determinista, pero en este caso su objetivo es validar identificadores. Este tipo de validación es muy común en los lenguajes de programación, donde los nombres de variables deben seguir ciertas reglas.

La regla implementada en este ejercicio es, un identificador debe comenzar con una letra y luego puede contener letras o números. Si el primer carácter es un número o un símbolo no permitido, la cadena se considera inválida.

Para representar esta lógica, el programa define un estado inicial que verifica el primer carácter de la cadena. Si ese carácter es una letra, el autómata pasa a un estado de aceptación. En ese nuevo estado, el autómata puede seguir leyendo letras o dígitos sin problema.

Si durante el proceso aparece un carácter que no corresponde a ninguno de los permitidos, el autómata pasa a un estado de rechazo. Esto significa que la cadena deja de ser válida como identificador.

La implementación utiliza una tabla de transiciones que indica cómo cambia el estado del autómata dependiendo del tipo de carácter leído. Cada carácter de la cadena se analiza uno por uno y se clasifica como letra, dígito u otro tipo de símbolo. Con base en esta clasificación se consulta la tabla de transiciones para determinar el siguiente estado.

3. Tercer punto, Calculadora utilizando Flex y Bison

El tercer programa corresponde a una calculadora desarrollada en lenguaje C utilizando las herramientas Flex y Bison. A diferencia de los programas anteriores, donde el análisis se implementó directamente en Python, en este caso el proceso se divide en dos etapas: el análisis léxico y el análisis sintáctico.

Flex se encarga de realizar el análisis léxico. Esto significa que lee el texto de entrada y reconoce los diferentes elementos que componen una expresión matemática, como números, operadores y saltos de línea. Cada uno de estos elementos se convierte en un token que posteriormente será utilizado por el parser.

Bison se encarga del análisis sintáctico. Su función es verificar que los tokens recibidos formen una expresión válida y, en caso afirmativo, ejecutar la operación correspondiente. De esta manera, la calculadora puede interpretar expresiones matemáticas escritas en un archivo de texto y calcular sus resultados.

La calculadora implementa las operaciones aritméticas básicas: suma, resta, multiplicación y división. Además, fue adaptada para trabajar con números reales, lo que permite realizar operaciones con valores decimales.

Otra característica importante es la implementación de la raíz cuadrada. En lugar de utilizar directamente una función matemática de la biblioteca estándar, se utilizó el método numérico de Newton-Raphson. Este método permite aproximar la raíz cuadrada de un número mediante un proceso iterativo. En cada iteración se calcula una nueva aproximación basada en la anterior, y el proceso continúa hasta que la diferencia entre dos aproximaciones consecutivas es suficientemente pequeña.

El programa también fue diseñado para leer las expresiones desde un archivo de texto. Esto permite escribir varias operaciones en un archivo y ejecutarlas todas en una sola corrida del programa. Cada línea del archivo contiene una expresión matemática, y la calculadora

imprime el resultado correspondiente en la consola.

4. Pruebas del programa

Para comprobar el funcionamiento de la calculadora se creó un archivo de texto que contiene varias expresiones matemáticas. En este archivo se incluyen ejemplos de suma, resta, multiplicación, división, raíz cuadrada y operaciones con números decimales.

5. Cuarto punto, algoritmo de Euclides en C y Haskell

En este punto se realizó una comparación entre dos implementaciones del algoritmo de Euclides para calcular el máximo común divisor (MCD). La comparación se hizo utilizando dos lenguajes con paradigmas diferentes: C, que sigue un enfoque imperativo, y Haskell, que utiliza un enfoque declarativo y funcional.

El algoritmo de Euclides es un método clásico para calcular el máximo común divisor entre dos números enteros. Su idea principal consiste en reemplazar repetidamente el par de números por el segundo número y el residuo de la división entre ambos, hasta que el segundo valor sea cero. En ese momento, el primer número corresponde al máximo común divisor.

5.1. Implementación en C

En C el algoritmo se implementa mediante una función recursiva que utiliza una condición para detener la ejecución cuando el segundo número llega a cero.

```
long long gcd(long long a, long long b) {  
    if (b == 0) return a;  
    return gcd(b, a % b);  
}
```

Este enfoque utiliza variables y control directo del flujo de ejecución, lo cual es característico del paradigma imperativo.

5.2. Implementación en Haskell

En Haskell la implementación se expresa mediante *pattern matching*, lo que permite escribir el algoritmo de forma muy similar a su definición matemática.

```
gcd' :: Integer -> Integer -> Integer  
gcd' a 0 = a  
gcd' a b = gcd' b (a `mod` b)
```

La principal diferencia es que Haskell describe el resultado de manera declarativa, en lugar de indicar paso a paso cómo debe ejecutarse el cálculo.

5.3. Resultados del benchmark

Para comparar el rendimiento de ambos lenguajes se realizaron pruebas ejecutando cada cálculo un millón de veces. Los programas fueron compilados con optimización utilizando -O2.

Par (a,b)	GCD	C (ns)	Haskell (ns)	Relación
(48,18)	6	7.66	16.1	×2.1
(100,75)	25	5.08	10.7	×2.1
(1071,462)	21	7.63	16.0	×2.1
(1000000,999999)	1	5.08	10.7	×2.1
(123456789,987654321)	9	7.61	16.0	×2.1
(999999937,999999893)	1	17.76	37.3	×2.1

También se realizó una prueba de estrés utilizando diez millones de pares aleatorios.

Lenguaje	Tiempo total	Promedio por llamada
C	863 ms	86.3 ns
Haskell	1812 ms	181.2 ns

Los resultados muestran que la implementación en C es aproximadamente 2.1 veces más rápida que la implementación en Haskell.

5.4. Conclusión de la comparación

A partir de los resultados obtenidos se puede concluir que C es la mejor opción cuando el objetivo principal es obtener el máximo rendimiento posible. Su ejecución es más cercana al hardware y tiene menor sobrecarga.

Por otro lado, Haskell ofrece una forma más clara y cercana a la definición matemática del algoritmo, además de permitir trabajar con números de precisión arbitraria sin necesidad de herramientas adicionales. Por esta razón, puede ser una mejor opción en contextos donde la claridad del código o la manipulación de números muy grandes sea más importante que la velocidad absoluta.